

SEPTEMBER 2026 SEMESTER

Lab06_OCR_Assignment

Project Name:

Students' Details:

NO.	NAME	STUDENT ID NO.
1	AIMAR ZUFAYRI BIN ZULFADLI	24006517

PART A:

Q1. NLP pipeline

Describe the five stages of an NLP pipeline: acquire, normalise, represent, model and evaluate. For each stage, give an example of what happens when a handwritten medicine-name image is processed.

Acquire: Obtaining raw text or data from inputs such as photos, audios or keyboards.

- Example for handwriting: Taking a digital photograph of the handwritten medicine name on a prescription slip.

Normalise: To convert text into a consistent format by handling case, punctuation and spacing.

- Example for handwriting: Converting the raw OCR output strings to lowercase first and stripping all the spaces and special symbols.

Represent: To translate text into a structured format that computers and algorithms can process and understand mathematically.

- Example for handwriting: Converting the now normalized medicine name string into token counts, n-grams or vector embeddings.

Model: Applying an algorithmic or machine learning model to execute a specific task like classification, translation or searching.

- Example for handwriting: Passing the feature representations or images through Tesseract LSTM neural-network engines to predict character sequences.

Evaluate: Comparing the system's output against a correct reference text using a performance metrics.

- Example for handwriting: In our lab, we measure the Character Error Rate (CER) or the Levenshtein distance between the OCR prediction and the true human-labelled medicine name.

Q2. How OCR works

Explain the difference between segmentation and character recognition. Give one example of a failure at each stage. Why is it useful to identify which stage caused an error?

Segmentation: Is the process of locating, bounding and isolating text regions, lines or individual characters within an image.

Character Recognition: The subsequent process of identifying those segmented shapes and mapping them into digital text of characters.

Example of Segmentation Failure is that text lines merging because the handwriting touches the edge of the image.

Example Recognition Failure is misidentifying a cursive letter "n" as a "u" because of overlapping strokes.

This is useful because it tells you where the system failed so you can the correct fix. For example, when we add borders to fix segmentation or using a handwriting-specific model to fix character recognition.

Q3. Text normalisation

The reference is "Tab Azimax 40mg" and the OCR output is "tab azimax 40 mg". Apply the Lab 6 normalisation rule (lowercase and keep only letters and digits). Show both normalised strings. Why must the same rule be applied to both?

Normalised Strings:

- Reference: "tab Azimax 40mg" → "tabazimax40mg"
- OCR output: "tab azimax 40 mg" → "tabazimax40mg"

The rules must be applied to ensure a fair comparison. If you clean one string differently from the other, variations like uppercase letters or spacing would cause false predictions during evaluation even if the actual characters read are correct.

Q4. Normalisation trade-off

The Lab 6 normaliser removes spaces. Describe one situation where removing spaces could hide an important difference. Suggest how you would change the normalisation rule for that situation.

Situation:
In a clinical setting , dosages and drug names are often adjacent. For examples “paracetamol500mg” and “paracetamold50mg”. Removing all spaces can blur distinct words or numerical boundaries.

Suggested change:

Modify the normalisation rule to preserve critical token boundaries or explicitly standardise whitespace rather than completely stripping them out.

PART B:

Q5. Levenshtein distance

Calculate the Levenshtein distance between “breeky” (reference) and “beeky” (prediction). Identify the edit operation(s). Then calculate the normalised edit distance by dividing the distance by the length of the longer string.

Calculation:

Reference (a): “breeky”

Prediction (b): “beeky”

Operation: 1 Deletion (removing one ‘e’)

Levenshtein Distance = 1

Normalise Edit Distance:

Length of longer string (N) = 6 (“breeky”)

Normalised Distance = 1/6

Q6. Character Error Rate (CER)

An OCR system produces 4 substitutions, 2 deletions and 1 insertion. The reference contains 35 characters. Calculate CER using CER = (S + D + I) / N. Explain what the result means. Can CER be greater than 1? Explain briefly.

Calculation:

- Substitutions (S) = 4
- Deletion (D) = 2
- Insertions (I) = 1
- Reference length (N) = 35
- CER = 0.20

Meaning: The OCR output required a 20% character-level correction relative to the total length of the reference text.

Q7. Compare the metrics

Explain exact-match accuracy, mean normalised edit distance, CER and blank predictions. For each metric, state whether a higher or lower value is generally better. Why is exact-match accuracy alone insufficient?

Exact-Match Accuracy: The percentage of images where the normalised prediction matches the reference identically. Higher is better.

Mean Normalised Edit Distance: The average ration of character edits divided by the longer string length. Lower is better.

Character Error Rate (CER): Total character edits divided by total reference characters. Lower is better.

Blank Predictions: The count of images where the OCR output is completely empty. Lower is better.

Why exact-match is insufficient: It is a binary. If an OCR system reads 9 out of 10 letters of a word correctly, exact-match treats it as a total failure (0%). Masking partial progress or usability.

PART C:

Q8. Read the results

Describe at least two improvements shown by the preprocessed results. Use values from the table as evidence.

1. Preprocessing significantly reduced blank predictions from 39 down to 15, indicating that padding and contrast enhancements helped the engine detect text presence more reliably.
2. The Character Error Rate improved after dropping from 0.875 to 0.788, proving that overall text recovery has improved.

Q9. Explain the result pattern

CER and mean normalised edit distance improved, and blank predictions decreased. However, exact-match accuracy increased only from 0.000 to 0.011. Explain how preprocessing can help recover text without making the complete medicine name correct.

Preprocessing fixes structural image issues like text touching borders or low contrast colours. This helps the OCR engine segment and recover characters it previously missed entirely. However, because Tesseract's recognition model is trained on printed fonts rather than cursive handwriting, it still misinterprets loops and connected cursive strokes, meaning characters are recovered as wrong letters as compared to it being actual correct ones.

Q10. Controlled experiment

Why did the lab keep the Tesseract settings (--oem 1 --psm 7), the 89 images and the normalisation rule unchanged? What would be difficult to conclude if several settings changed at once?

Keeping Tesseract configurations (--oem 1 –psm 7), the exact 89 images and the normalisation rules completely fixed ensures a controlled experiment. If multiple settings changed at once, it would be scientifically impossible to conclude whether performance changes were caused by your image preprocessing steps, altered engine settings, or different dataset subsets.

Q11. Diagnose the failure

Identify the likely category for each case and explain:

(a) Tesseract raises an error while processing an image;
Engine Error. Tesseract crashed or threw a runtime execution exception while attempting to parse an invalid or corrupted image file.

(b) OCR returns an empty string;
No Text Detected. The handwriting was too faint, low-contrast, or poorly segmented, leading the engine to find no readable text line

(c) OCR returns “nidu” when the reference is “nims”.
Character Confusion. The engine successfully detected text but misread specific handwritten characters (substituting “m” for “d” and “s” for “u”) due to similarities in cursive strokes.

PART D:

Q12. Suggest improvements

Suggest TWO improvements you would investigate next to improve handwritten OCR. For each, explain which error it targets and how you would test whether it helps. Do not claim an improvement works without evidence.

Improvement 1: Use a handwriting-specific Deep Learning OCR model (like CRNN or TrOCR) trained on cursive scripts.

Target error: Character confusion and poor cursive recognition.

How to test: we can train/evaluate the model on a held-out split of the prescription dataset and compare its CER and exact accuracy against Tesseract's baseline.

Improvement 2: Advanced binarization and thresholding. This is to cleanly separate handwriting ink from noise backgrounds and lined paper.

Target error: No text detected/faint handwriting failures.

How to test: We run a controlled experiment measuring the drop in the blank predictions before and after applying binarization.

Q13. Medical safety scenario

A clinic wants to use OCR to read medicine names and dosages from handwritten prescriptions. Explain at least THREE risks of using OCR output without human checking. Recommend a safer workflow.

Incorrect Drug Dosage: Misreading numbers or units (e.g., reading 5mg as 50mg) can lead to toxic overdoses.

Wrong Medication Name: Character confusion can swap a benign drug name for a dangerous contraindication.

Patient Harm/Liability: Automated transcription errors can cause severe adverse drug reactions or fatal clinical mistakes.

Q14. Reflection

What is the most important thing you learned from Lab 6? Identify one concept you initially found difficult and explain it using an example from the practical.

The Most Important Lesson: Automated OCR outputs on unstructured, real-world cursive data are inherently unreliable and unsafe for high-stake domains without rigorous preprocessing and strict human oversight.

Difficult Concept & Explanation: Character Error Rate (CER) exceeding 1.0. Initially, it felt counter intuitive that and error rate could go above 100%. However, understanding the formula which clarifies that if an engine hallucinates and outputs a long string of extra characters, the total edit count can exceed the actual length of the reference string N, driving the ration past 1.0.